



Hướng dẫn toàn diện: Async, Await & Task trong C# / .NET

Tài liệu này tổng hợp toàn bộ lý thuyết cốt lõi về lập trình bất đồng bộ (**Asynchronous Programming**) trong C# / .NET, đi kèm các ví dụ minh họa thực tế và phân tích trực tiếp từ codebase của dự án [School_CRUD_console](file:///home/van/dev/DotNet/School_CRUD_console).

1. Ví dụ thực tế dễ hiểu (Real-world Analogy)

Hãy hình dung bài toán thông qua **Mô hình Quán Cà Phê**:

[Khách hàng] ---> [Nhân viên thu ngân] ---> [Máy pha cà phê / Ổ đĩa cứng I/O]

● Lập trình Đồng bộ (Synchronous - Sync)

- Thu ngân nhận đơn từ Khách A → Đứng chờ máy pha cà phê làm xong (10 phút) → Trao ly cho Khách A → Mới tiếp tục nhận đơn của Khách B.
- Hậu quả:** Thu ngân bị "khóa cứng" (**Thread Blocked**). Khách B phải đứng chờ dù thu ngân không làm gì ngoài việc... đứng đợi.

● Lập trình Đa luồng (Multithreading)

- Quán thuê thêm 10 thu ngân để phục vụ 10 khách cùng lúc.
- Hậu quả:** Tốn rất nhiều chi phí trả lương (**Tốn tài nguyên RAM, CPU Context Switching** giữa các Thread).

● Lập trình Bất đồng bộ (Asynchronous - Async/Await)

- Thu ngân nhận đơn từ Khách A → Gửi lệnh cho máy pha → Trả lại thẻ rung

(giao `Task`) cho Khách A → **Ngay lập tức nhận đơn của Khách B.**

- Khi máy pha xong, thẻ rung báo (`await`), thu ngân quay lại trao ly cà phê cho Khách A.
- **Tác dụng:** Chỉ cần 1 thu ngân (**1 Thread**) vẫn phục vụ mượt mà hàng trăm khách hàng mà không ai phải chờ vô ích (**Non-blocking I/O**).

2. Các Khái niệm Cốt lõi (Core Concepts)

A. `Task` và `Task<T>`

`Task` đại diện cho một **công việc đang hoặc sẽ thực thi trong tương lai.**

Kiểu trả về	Tương đương trong Sync	Ý nghĩa
<code>Task</code>	<code>void</code>	Công việc bất đồng bộ không trả về giá trị (chỉ chờ xong).
<code>Task<T></code>	<code>T</code> (ví dụ <code>int</code> , <code>string</code> , <code>Student</code>)	Công việc bất đồng bộ có trả về kết quả kiểu <code>T</code> .
<code>Task.CompletedTask</code>	N/A	Trả về một <code>Task</code> đã hoàn thành ngay lập tức (dùng khi logic bên trong là đồng bộ).

B. Từ khóa `async` và `await`

- **`async`** : Đánh dấu một phương thức có chứa thao tác bất đồng bộ. Từ khóa này báo cho C# Compiler tạo ra một **State Machine (Máy trạng thái)** ngầm để quản lý tiến trình.
- **`await`** : Nhường quyền điều khiển Thread cho hệ thống trong khi chờ thao tác I/O (File, Network, Database) hoàn thành. Code bên dưới từ khóa `await` sẽ được tiếp tục chạy sau khi `Task` hoàn thành.

3. Các Cạm bẫy cần tránh (Common Pitfalls)

❌ 1. Cạm bẫy `async void`

Quy tắc vàng: Không bao giờ viết `public async void MethodName()`, trừ khi đó là Event Handler của UI (như `button_Click`).

- **Lý do:**

- i. Caller không thể dùng `await` đối với hàm `async void`.
- ii. Nếu có ngoại lệ (Exception) xảy ra bên trong `async void`, nó không thể bị bắt bằng `try-catch` ở bên ngoài và sẽ làm **crash ứng dụng ngay lập tức**.

- **Cách khắc phục:** Đổi return type thành `async Task`.

```
// ❌ SAI (Không thể await, gây crash app nếu có lỗi)
private async void WriteLog(string msg) { ... }

// ✅ ĐÚNG
private async Task WriteLog(string msg) { ... }
```

🌊 2. Hiệu ứng Lây lan Async (Async Waterfall / Infection)

Khi một phương thức ở tầng sâu (ví dụ ghi log file `JSON_Logger`) chuyển sang `async Task`, từ khóa `await` sẽ yêu cầu tất cả các hàm gọi nó ở tầng trên cũng phải chuyển thành `async Task`:

```
JSON_Logger.LogInfo() [async Task]
├─ CompositeLogger.LogInfo() [async Task]
│   └─ Semester.RunAsync() [async Task]
│       └─ School.StartSimulationAsync() [async Task]
│           └─ Program.cs [await Top-level statement]
```

4. Phân tích Ví dụ Thực tế trong Project School_CRUD_console



Ví dụ 1: Ghi log Async thực sự với File.AppendAllTextAsync

File: [JSON_Logger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/Interface_applied/Log/JSON_Logger.cs#L29-L48)

```
public class JSON_Logger : ILogger
{
    private readonly string _logfilepath = "./JSON Log/log.json";

    private async Task WriteLog(string msg, string level)
    {
        var options = new JsonSerializerOptions
        {
            WriteIndented = false,
            Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping, // Giữ UTF-8 cho tiếng
            PropertyNameCaseInsensitive = true
        };

        JSONItem item = new(msg, level);
        string jsonString = JsonSerializer.Serialize(item, options);

        // 🟢 Bất đồng bộ thực sự: Nhường Thread trong lúc HĐH ghi dữ liệu xuống đĩa cứng
        await File.AppendAllTextAsync(_logfilepath, jsonString + Environment.NewLine);
    }

    public async Task LogInfo(string msg) => await WriteLog(msg, "INFO");
    public async Task LogWarning(string msg) => await WriteLog(msg, "WARNING");
    public async Task LogError(string msg) => await WriteLog(msg, "ERROR");
    public async Task LogSuccess(string msg) => await WriteLog(msg, "SUCCESS");
}
```

- **Giải thích:**

- i. `File.AppendAllTextAsync` là hàm bất đồng bộ do .NET cung cấp. Khi gọi `await File.AppendAllTextAsync(...)`, Thread hiện tại được giải phóng để làm việc khác trong lúc chờ ổ đĩa SSD/HDD ghi dữ liệu xong.
- ii. `JSONItem` phải khai báo các `public property` (`Message`, `Level`, `Timestamp`) thì `JsonSerializer` mới serialize thành công.

Ví dụ 2: Đồng bộ đóng giả Bất đồng bộ với `Task.CompletedTask`

File: [ConsoleLogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/Interface_applied/Log/ConsoleLogger.cs#L10-L35)

```
public class ConsoleLogger : ILogger
{
    public Task LogInfo(string msg)
    {
        Console.WriteLine($"INFO: [{date}], {msg}");
        // 🟢 Thỏa mãn Interface Task mà không cần từ khóa async/await
        return Task.CompletedTask;
    }
}
```

- **Giải thích:**

- Thao tác `Console.WriteLine` là thao tác CPU/Bộ nhớ đồng bộ.
- Do [ILogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/ILogger.cs) quy định hợp đồng trả về `Task`, `ConsoleLogger` chỉ cần trả về `Task.CompletedTask` để báo cho caller biết "Task này đã xong ngay lập tức", giúp tránh tốn chi phí tạo State Machine của từ khóa `async`.

Ví dụ 3: Ủy quyền Await trong Mẫu Thiết kế Composite

File: [CompositeLogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/

```
public class CompositeLogger : ILogger
{
    private readonly List<ILogger> _listLogger = new List<ILogger>();

    public async Task LogInfo(string msg)
    {
        foreach (var logger in _listLogger)
        {
            // 🟢 Await từng logger thành phần (ConsoleLogger, JSON_Logger)
            await logger.LogInfo(msg);
        }
    }
}
```

- **Giải thích:**

- `CompositeLogger` duyệt qua danh sách các logger con và `await` từng đợt ghi log của từng logger.



Ví dụ 4: Vòng đời gọi Async ở Main Entry

File: [Program.cs](file:///home/van/dev/DotNet/School_CRUD_console/Program.cs#L15-L25)

```
ILogger consoleLog = new ConsoleLogger();
ILogger JsonLog = new JSON_Logger();
CompositeLogger compositeLog = new(consoleLog, JsonLog);

IGPACalculator GPACalc = new GPA4Calculator();
School school = new School(compositeLog, GPACalc);

// 🟢 Top-level statements hỗ trợ await trực tiếp ở hàm Main
await school.InitializeDataAsync(20);
await school.StartSimulationAsync(2);
```

5. Bảng Tổng kết Quy tắc sử dụng Async / Await

Tình huống	Nên dùng	Ví dụ
Thao tác I/O (File, DB, Web API)	<code>async Task /</code> <code>async Task<T> + await</code>	<code>await File.AppendAllTextAsync(...)</code>
Thao tác CPU/ Memory thuần túy	Trực tiếp <code>Task.CompletedTask</code> hoặc <code>Task.FromResult(val)</code>	<code>return Task.CompletedTask;</code>
Khai báo Interface Async	<code>Task LogAsync(string msg);</code>	<code>[ILogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/ILogger.cs)</code>
Gọi phương thức Async	Luôn luôn <code>await</code>	<code>await logger.LogInfo(...)</code>
Khai báo Return Type	Tránh <code>async void</code> (Trừ UI Event Handler)	Đổi tất cả <code>async void</code> → <code>async Task</code>

Ran command: `ls`

Ran command: `git init`

Ran command: `dotnet new gitignore`

Ran command: `git add .`

Ran command:

```
git commit -m "first commit: Finish version 3 of project, apply async await in Logger"
Viewed JSON_Logger.cs:29-49
Edited Async_Await_Guide.md
```

Tôi đã tạo thành công tài liệu tổng hợp kiến thức đầy đủ về **Async, Await & Task** trong **C# / .NET** tại file [Async_Await_Guide.md](file:///home/van/dev/DotNet/School_CRUD_console/Async_Await_Guide.md).



Tóm tắt các nội dung chính trong tài liệu:

1. Ví dụ thực tế dễ hiểu (Quán Cà Phê Analogy):

- **Sync (Đồng bộ):** Thu ngân đứng chờ pha cà phê xong mới tiếp khách tiếp theo (gây nghẽn Thread).
- **Multithreading:** Thuê thêm 10 thu ngân (tốn tài nguyên RAM, CPU Context Switch).
- **Async/Await:** Thu ngân nhận đơn → giao thẻ rung cho khách → tiếp tục phục vụ khách khác → khi thẻ rung (`await`) thì quay lại giao cà phê (**Non-blocking I/O**).

2. Khái niệm cốt lõi:

- **Task & Task<T>** : Đại diện cho công việc thực thi trong tương lai.
- **Task.CompletedTask** : Dùng cho thao tác đồng bộ nhưng cần tuân thủ Interface bất đồng bộ (như trong [ConsoleLogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/Interface_applied/Log/ConsoleLogger.cs)).
- **async & await** : Đánh dấu State Machine và điểm nhường Thread cho hệ thống.

3. Cạm bẫy cần tránh:

- **Tuyệt đối tránh `async void`** : Ngoại trừ Event Handler của UI, vì không `await` được và sẽ làm crash app nếu có ngoại lệ không được catch.
- **Lây lan Async (Async Waterfall):** Khi tầng sâu (`JSON_Logger`) thành `async Task`, các tầng trên (`Semester`, `School`, `Program`) cũng phải đổi thành `async Task` và `await`.

4. Phân tích chi tiết code trong Project:

- Phân tích [JSON_Logger.cs](file:///home/van/dev/DotNet/

School_CRUD_console/Interfaces/Interface_applied/Log/JSON_Logger.cs):
Ghi file bất đồng bộ thực sự với `File.AppendAllTextAsync` .

- Phân tích [ConsoleLogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/Interface_applied/Log/ConsoleLogger.cs):
Dùng `Task.CompletedTask` để tối ưu chi phí State Machine.
- Phân tích [CompositeLogger.cs](file:///home/van/dev/DotNet/School_CRUD_console/Interfaces/Interface_applied/Log/CompositeLogger.cs): Duyệt mảng và `await` đa logger.
- Phân tích [Program.cs](file:///home/van/dev/DotNet/School_CRUD_console/Program.cs): Gọi `await` ở Top-level statements.